CMPT 125-D100 Midterm Exam
Spring 2026

Solutions

This is a **50 minute closed-book exam**: notes, books, computers, calculators, electronic devices, etc. are **not** permitted. Do not speak to any other students during the exam or look at their work. Please remain seated and **raise your hand** if you have a question.

Multiple Choice

Each correct answer is worth 1 mark. Incorrect answers, or unanswered questions, are worth 0. Make sure to fill in your answer on the answer sheet at the end of this exam.

1) How many * characters does this print?

```
string s = "4031";
for (char c : s) {
    switch (c) {
        case '0':
            cout << '*';
        case '1':
            cout << '*';
        default:
            cout << '*';
    }
}
```

A. 4

B. 5

C. 6

D. 7

E. 8

D, without break the case statements "fall through"
*******=7

2) Consider this function:

```
int recap(string& s, string t) {
  // ...
}
```

i) `s` is passed by reference
ii) `t` is passed by value

A. i) and ii) are both true

B. i) and ii) are both false

C. i) is false and ii) is true

D. i) is true and ii) is false

A

3) What does this program print?

A. `a`
B. `m`
C. `\0`
D. either some garbage value, or it crashes at runtime
E. nothing: there's a compiler error

```
#include <iostream>

using namespace std;

int main() {
    const char* s = "spam";
    cout << *(s + 3);
}
```

Solution: b)
This is a correct program that prints the letter m.

Solutions

4) In which memory region are local variables and function parameters typically stored?	A. Free store memory B. Read-only memory C. Call stack memory D. Static memory

Solution: c)

5) This program has a memory leak.
A. True B. False
 	
6) This program has a dangling pointer.
A. True B. False
 	
7) This program prints 5.
A. True B. False
 	
8) In this program the type of `&p` would be `int*`
A. True B. False
	`#include <iostream>`
	
	`using namespace std;`
	
	`int main() {`
	`  int n = 5;`
	`  int* p = &n;`
	`  cout << *p;`
	`}`

5. b) False, all the memory is on the stack and so automatically de-allocated
6. b) False, p is the only pointer and it always points to valid memory
7. a) True
8. b) False, p is of type int*, so & would be type int**

9) This program has a memory leak which can be fixed by putting `delete p;` after the `cout` statement in `main`.
A. True B. False
	`#include <iostream>`
	
	`using namespace std;`
	
	`int* twice(int x) {`
	`  int result = 2 * x;`
	`  return &result;`
	`}`
	
	`int main() {`
	`  int* p = twice(3);`
	`  cout << *p;`
	`}`

Solution: b) False. There is no memory leak. Instead, the problem is that `twice` returns a pointer to the local variable `result`. Since local variables are on the call stack, `result` is popped when `twice` ends, meaning `p` is assigned an invalid address.

Solutions

10) What does this program print?
A. 2 2
B. 2 5
C. 5 2
D. 5 5
E. nothing: it crashes at run-time

```
#include <iostream>

using namespace std;

void f(int* a, int* b) {
  int temp = *a;
  *b = *a;
  *a = temp;
}

int main() {
  int x = 2;
  int y = 5;
  f(&x, &y);
  cout << x << " " << y;
}
```

Solution: a). By carefully tracing the program (using diagrams to help), you can determine the printed value for x is 2 and for y is 2.

11) What does this program print?
A. 012
B. 123
C. three unknown `int` values
D. nothing: it has an error

12) `arr` is correctly de-allocated and the program has no memory errors.
A. True B. False

```
#include <iostream>

using namespace std;

int main() {
    int* arr = new int[3];
    for(int* p = arr; p < arr + 3; p++) {
        *p = p - arr + 1;
    }
    cout << arr[0] << arr[1] << arr[2];

    delete[] arr;
}
```

11) B) 012
12) A) True

13) Consider these statements:

i) An advantage of `assert`-style testing over plain if-statement testing is that `asserts` tell you the line in the source code that failed.

ii) Table-based testing makes it relatively easy to add and remove tests.

A. i) and ii) are both true

B. i) and ii) are both false

C. i) is false and ii) is true

D. i) is true and ii) is false

A

Solutions

14) Consider these statements:

i) System testing is a kind of whitebox testing.

ii) Whitebox testing is a kind of blackbox testing.

A. i) and ii) are both true
B. i) and ii) are both false
C. i) is false and ii) is true
D. i) is true and ii) is false

B

15) Consider these statements:

i) If `p == nullptr`, then evaluating `*p` is always an error.

ii) Any pointer variable can be assigned `nullptr`.

A. i) and ii) are both true
B. i) and ii) are both false
C. i) is false and ii) is true
D. i) is true and ii) is false

A

16) Consider this (correctly working) code fragment:

```
string* p = new string("SFU");
```

i) `*p.size()` returns 3
ii) `p->size()` returns 3

A. i) and ii) are both true
B. i) and ii) are both false
C. i) is false and ii) is true
D. i) is true and ii) is false

C

Solutions

17) What is a correct implementation of `operator!=` ?

```
class MyString {
   // ...
public:
   // ...

   bool operator==(const MyString& other) const {
     // ... code to check if this MyString and other MyString
     //     have the same chars in the same order ...
   }

   bool operator!=(const MyString& other) const {
     // ...
   }

};
```

A. `return this != &other;`
B. `return !(==(other));`
C. `return !(*this == other);`
D. `return !(this == other);`

C

18) Suppose class `Cow` and class `Pig` both inherit from class `Animal`.

i) A pointer of type `Cow*` can point to an object of type `Pig`.

ii) A pointer of type `Animal*` can point to an object of either type `Cow` or type `Pig`.

A. i) and ii) are both true

B. i) and ii) are both false

C. i) is false and ii) is true

D. i) is true and ii) is false

C

Solutions

19) Consider this code:

```
class Menu {
public:
   string get_title() const {
       return "Menu";
   }
};

class Fancy_menu : public Menu {
public:
    string get_title() const {
       return "Fancy Menu";
    }
};
```

And also:

```
Menu* a = new Menu();        // line 1
Menu* b = new Fancy_menu(); // line 2
```

A. `a->get_title()` returns `"Menu"`, and `b->get_title()` returns `"Fancy Menu"`
B. `a->get_title()` returns `"Menu"`, and `b->get_title()` returns `"Menu"`
C. line 2 causes a compile-time error

B

20) Suppose that class `Child` publicly inherits from `Base`, e.g.:

```
class Child : public Base {
  // …
};
```

What does **`public`** mean in this situation?

A. `Child` only inherits the *public* methods and variables of `Base`
B. `Child` only inherits the *private* methods and variables of `Base`
C. all the methods and variables inherited from `Base` are treated as *public* in `Child`
D. all the methods and variables inherited into `Child` keep the same visibility as they had in `Base`, i.e. public members stay public, private members stay private

D

Solutions

21) What is `std` in the C++ standard library?	A. an exception B. a namespace C. a function D. a class
B	

22) What is an *exception* in C++?	A. An object that represents an exceptional situation, such as a bad error. B. A function that triggers an error. C. A bad or unexpected value on the call stack. D. The general name for any error that occurs while compiling a program.
A	

23) Namespaces are necessary: i) for separate compilation ii) if you want to write your own `.h` files to `#include` in programs	A. i) and ii) are both true B. i) and ii) are both false C. i) is false and ii) is true D. i) is true and ii) is false
B	

24) What happens if an exception is thrown but not caught?	A. Memory is corrupted. B. A compiler error: all exceptions must be caught. C. Nothing, the exception is ignored. D. The program crashes at run time.
D	

25) What does `f(2)` print?
A. `2121`
B. `2211`
C. `2212`
D. `2112`
	`void f(int n) {`
	`  if (n == 0) {`
	`  return;`
	`  } else {`
	`  cout << n;`
	`  f(n - 1);`
	`  cout << n;`
	`  }`
	`}`

D. 2112	

Solutions

26) If `v` is a `vector<int>`, what does `f(v, 0)` return?

A. the sum of the odd numbers in `v`
B. the sum of the even numbers in `v`
C. the count of the odd numbers in `v`
D. the count of the even numbers in `v`
E. it returns an int, but not any of the above

```
int f(const vector<int>& v, int i) {
    if (i >= v.size()) {
        return 0;
    } else {
        int r = v[i] % 2;
        return v[i] * r + f(v, i + 1);
    }
}
```

A. the sum of the odd numbers

Solutions

Object-oriented Programming and Inheritance

a) (6 marks) Add the following to the `Movie` class below:

1. A **copy constructor**.
2. A **destructor** that prints "done!".
3. A **setter** that lets the user change the name of a `Movie`.
4. A **getter** that returns the year of a `Movie`.
5. `==` that tests if two `Movie` objects have the same title and year.

```
class Movie {
    string title;
    int year;

public:
    Movie(const string& t, int y) {
       title = t;
       year = y;
    }

    // ... your code goes here ...
```

Sample Solution

```
class Movie {

// ...

Move(const Movie& m) {
   title = m.title;
   year = m.year;
}
// Movie(const Movie& m)
// : title(m.title), year(m.year) { }
// Movie(const Movie& m)
// : Movie(m.title, m.year) { }

~Movie() { cout << "done!"; }

void set_title(const string& t) { title = t; }

int get_year() const { return year; }

bool operator==(const Move& other) const {
   return title == other.title && year == other.year;
}

}; // class Movie
```

Solutions

Memory Management

(6 marks) Suppose you are making a dynamic array of integers based on this struct:

```
struct List {
   int* arr; // pointer to underlying array
   int sz;   // number of elements in list
   int cap;  // size of underlying array
};
```

Write a `void` function called **`enlarge`** that takes a `List` as input and modifies it so that its capacity is increased by a factor of 1.75.

Both the size and the `int`s on the list are unchanged. You can assume that the list is valid and non-empty, and that `arr` points to an array that was allocated with `new`.

For example, suppose `lst` is initialized like this:

```
List lst = {new int[10], 3, 10};
lst.arr[0] = 1;
lst.arr[1] = 2;
lst.arr[2] = 3;
```

Then after calling `enlarge(lst)`, `lst` has the same size and `int`s, but now has a new capacity of `int(1.75 * 10) = int(17.5) = 17`. The underlying array is 17 `int`s long, and the first three elements are still 1, 2, and 3.

Important: **Don't** use any #includes or anything from the C++ standard library (like vectors or algorithms).

Solution

```
// lst is passed by reference
void enlarge(List& lst) {
    int new_cap  = lst.cap * 1.75;   // calculate new capacity
    int* new_arr = new int[new_cap]; // allocate new array
    for (int i = 0; i < lst.sz; i++) // copy old array
        new_arr[i] = lst.arr[i];

    delete[] lst.arr;  // delete old array
    lst.arr = new_arr; // assign new array
    lst.cap = new_cap; // update capacity
}
```

Solutions